重庆大学大数据与软件学院

作 业 报 告

实践项目	ATM 模拟编程作业
课程名称	计算机网络

姓名	沈双双	学号	20230788
姓名	谢丽欣	学号	20230789
教师	胡海波	班级	094003-002
日期	2025.05.25	成绩	

1. 目的

通过作业二使同学们掌握所学 TCP 套接字编程方法，理解 TCP 套接字在服务器端和客户端之间的应用层通信的流程和形式，巩固计算机网络的理论和知识。此外，通过作业的开展使同学们提高计算机网络程序设计和开发能力，熟悉所用编程语言的开发环境及调试过程，熟悉所用语言中的数据类型、数据结构、语法结构、运算方法等。基于作业一的内容进行程序设计开发，同学们也可以体会计算机网络通信协议(应用层)的制定、标准化等工作对实际应用的价值和意义。

2. 作业内容

要求两位同学组队(参考石墨文档 1-作业 2 分组)，基于模拟的应用层通信协议标准(参考石墨文档 2-“RFC20232023”)完成 ATM 机客户端和服务器端的程序设计和开发。程序语言不限。

客户端具备 GUI 可以模拟 ATM 机的账户登录和存款取款等功能，同时应该包括错误信息的提示。服务器端应该需要通过后台数据库维护账户信息、具备记录日志的功能。

各组同学独立完成，不能抄袭。作业结束时，每组需提交作业报告，其中包含程序设计思路和软件源代码。

里程碑 1：第 2 次实验课，完成组内程序设计开发和测试，尝试连接其他小组的服务器端。

里程碑 2：第 3 次实验课，分析解决里程碑 1 中的问题，完成程序调试，通过测试脚本完成对其他所有小组服务器端的连接，记录结果。

里程碑 3：第 4 次实验课，继续调试程序，最终提交文档、代码。

3. 运行环境

1.**系统环境**：Windows 11

2.**编码环境**: IntelliJ IDEA Community Edition 2024.3.4.1

3.**程序语言**:Java11.0.17

4. **相关知识**：TCP Socket 是一种基于 TCP/IP 协议的网络通信机制，用于实现不同计算机之间的数据 交换。其基本原理包括以下几个方面：
①IP 地址和端口号：每台计算机都有唯一的 IP 地址，用于标识不同的计算机。而端口 号则用于区分一个计算机上不同的进程或应用程序。
②三次握手建立连接：在进行 TCP Socket 通信之前，需要通过三次握手协议来建立连 接。这个过程包括客户端向服务器端发送请求，服务器端接受请求并回复确认，最后客户端再回复确认。
③数据传输：TCP Socket 会将要传输的数据进行分段，并且对每个分段进行编号和校 验，以保证数据的可靠性和完整性。同时，还会对数据进行流量控制和拥塞控制，确保网络 质量和带宽合理利用。
④四次挥手断开连接：当数据传输完成后，需要通过四次挥手协议来断开连接。这个过 程包括客户端向服务器端发送请求断开连接，服务器端回复确认，服务器端向客户端发送断开连接 请求，客户端回复确认，最终完成断开连接的操作。

4. 程序设计

4.1 设计思路

RFC-20232023

1. Messages from ATM to Server

Msg name	Purpose
HELO sp <userid>	Let server know that there is a card in the ATM machine
	ATM transmits user ID (cardNo.) to Server
PASS sp <passwd>	User enters PIN (password), which is sent to server
BALA	User requests balance
WDRA sp <amount>	User asks to withdraw money
BYE	user all done

2. Messages from Server to ATM

Msg name	Purpose
500 sp AUTH REQUIRE	Ask user for PIN (password)
525 sp OK!	Requested operation (PASSWD, WITHDRAWL) OK
401 sp ERROR!	requested operation (PASSWD, WITHDRAWL) in ERROR
AMNT:<amnt>	Sent in response to BALANCE request
BYE	User done, display welcome screen at ATM

---

3. Interaction between ATM and Server:

Client **Server**

HELO <userid> ----------> (check if valid userid)

<---------- 500 sp AUTH REQUIRED!

PASS <passwd> ----------> (check password)

<---------- 525 OK! (//password is OK)

BALA ----------> (check balance from database)

<---------- AMNT:<amnt>

WDRA <amnt> ----------> (check if enough money to cover withdrawal)

<---------- 525 OK (if enough, update database)

(ATM dispenses)

<---------- 401 sp ERROR! (else)

BYE ---------->

<---------- BYE

4. Server port number

2525

4.2 服务器端

ATMServer.java 中：

```
import java.io.*;
import java.net.*;
import java.sql.*;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

/**
 * ATM 服务器端主类，处理客户端连接和 ATM 操作请求
 */
public class ATMServer {
    // 服务器端配置常量
    private static final int PORT = 2525;                    // 服务器端
监听端口
    private static final String DB_URL =
"jdbc:sqlite:F:/soft/SQlite/atm.db";  // 数据库连接 URL

    // 线程池，用于处理客户端连接
    private static final ExecutorService threadPool =
Executors.newCachedThreadPool();

    /**
     * 主方法，启动服务器端
     */
    public static void main(String[] args) throws IOException,
SQLException {
        // 创建服务器端 Socket
        ServerSocket serverSocket = new ServerSocket(PORT);
        System.out.println("Server started. Listening on port " +
PORT);

        // 加载 SQLite JDBC 驱动
        try {
            Class.forName("org.sqlite.JDBC");
        } catch (ClassNotFoundException e) {
            System.out.println("SQLite JDBC driver not found.");
            e.printStackTrace();
            return;
        }

        // 主循环，持续接受客户端连接
        while (true) {
            Socket clientSocket = serverSocket.accept();
```

```
            System.out.println("New client connected: " +
clientSocket.getInetAddress());

            // 使用线程池处理每个客户端连接
            threadPool.execute(new ClientHandler(clientSocket));
        }
    }

    /**
     * 客户端请求处理类，实现 Runnable 接口以便在独立线程中运行
     */
    private static class ClientHandler implements Runnable {
        private final Socket clientSocket;  // 客户端 Socket 连接

        public ClientHandler(Socket socket) {
            this.clientSocket = socket;
        }

        @Override
        public void run() {
            // 使用 try-with-resources 确保流和 Socket 正确关闭
            try (BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
                 PrintWriter out = new
PrintWriter(clientSocket.getOutputStream(), true)) {

                String request;           // 客户端请求命令
                String username = null;    // 当前会话用户名
                boolean isAuthenticated = false;  // 认证状态标志

                // 读取并处理客户端请求
                while ((request = in.readLine()) != null) {
                    System.out.println("Received command: " + request);

                    // 处理 HELO 命令 - 用户名验证
                    if (request.startsWith("HELO")) {
                        username = request.split(" ")[1];
                        if (authenticateUser(username)) {
                            out.println("500 AUTH REQUIRE");  // 需要密码认
证
                        } else {
                            out.println("401 ERROR!");       // 用户名不存在
                            logServiceRecord(username, "HELO", "401
ERROR!");
```

```
                    break;
                }
            }
            // 处理 PASS 命令 - 密码验证
            else if (request.startsWith("PASS")) {
                String password = request.split(" ")[1];
                if (isAuthenticated = authenticateUser(username,
password)) {
                    out.println("525 OK!");          // 认证成功
                    logServiceRecord(username, "PASS", "525
OK!");
                } else {
                    out.println("401 ERROR!");       // 认证失败
                    logServiceRecord(username, "PASS", "401
ERROR!");
                    break;
                }
            }
            // 处理 BALA 命令 - 查询余额
            else if (request.equals("BALA")) {
                if (isAuthenticated) {
                    out.println("AMNT:" + getBalance(username));
// 返回余额
                    logServiceRecord(username, "BALA", "AMNT:" +
getBalance(username));
                } else {
                    out.println("401 ERROR!");       // 未认证
                    logServiceRecord(username, "BALA", "401
ERROR!");
                }
            }
            // 处理 WDRA 命令 - 取款操作
            else if (request.startsWith("WDRA")) {
                if (isAuthenticated) {
                    double amount =
Double.parseDouble(request.split(" ")[1]);
                    String response = withdraw(username, amount);
// 执行取款
                    out.println(response);
                    logServiceRecord(username, "WDRA", response);
                } else {
                    out.println("401 ERROR!");       // 未认证
                    logServiceRecord(username, "WDRA", "401
ERROR!");
```

```
                    }
                }
                // 处理 BYE 命令 - 结束会话
                else if (request.equals("BYE")) {
                    out.println("BYE");
                    logServiceRecord(username, "BYE", "BYE");
                    break;
                }
                // 未知命令处理
                else {
                    out.println("401 ERROR!");
                    logServiceRecord(username, "Unknown", "401
ERROR!");
                    break;
                }
            }
        } catch (Exception e) {
            System.out.println("Client disconnected
unexpectedly.");
            e.printStackTrace();
        } finally {
            try {
                clientSocket.close();  // 确保关闭客户端连接
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }

    /**
     * 验证用户名是否存在
     * @param username 要验证的用户名
     * @return 存在返回 true，否则 false
     */
    private static boolean authenticateUser(String username) throws
SQLException {
        try (Connection conn = DriverManager.getConnection(DB_URL);
             PreparedStatement stmt = conn.prepareStatement("SELECT *
FROM users WHERE username = ?")) {
            stmt.setString(1, username);
            ResultSet rs = stmt.executeQuery();
            return rs.next();  // 是否有结果
        }
```

```
    }

    /**
     * 验证用户名和密码是否匹配
     * @param username 用户名
     * @param password 密码
     * @return 匹配返回true，否则false
     */
    private static boolean authenticateUser(String username, String
password) throws SQLException {
        try (Connection conn = DriverManager.getConnection(DB_URL);
             PreparedStatement stmt = conn.prepareStatement("SELECT *
FROM users WHERE username = ? AND password = ?")) {
            stmt.setString(1, username);
            stmt.setString(2, password);
            ResultSet rs = stmt.executeQuery();
            return rs.next();  // 是否有结果
        }
    }

    /**
     * 获取用户余额
     * @param username 用户名
     * @return 用户余额
     */
    private static double getBalance(String username) throws
SQLException {
        try (Connection conn = DriverManager.getConnection(DB_URL);
             PreparedStatement stmt = conn.prepareStatement("SELECT
balance FROM users WHERE username = ?")) {
            stmt.setString(1, username);
            ResultSet rs = stmt.executeQuery();
            rs.next();
            return rs.getDouble("balance");
        }
    }

    /**
     * 执行取款操作
     * @param username 用户名
     * @param amount 取款金额
     * @return 操作结果代码
     */
    private static String withdraw(String username, double amount)
```

```
throws SQLException {
        try (Connection conn = DriverManager.getConnection(DB_URL)) {
            // 更新用户余额
            String sql = "UPDATE users SET balance = balance - ? WHERE
username = ?";
            try (PreparedStatement stmt = conn.prepareStatement(sql)) {
                stmt.setDouble(1, amount);
                stmt.setString(2, username);
                int updatedRows = stmt.executeUpdate();

                if (updatedRows > 0) {
                    // 记录取款操作
                    String insertSql = "INSERT INTO service_records
(username, service_name, amount, response_code, timestamp) VALUES
(?, ?, ?, '525 OK', datetime('now'))";
                    try (PreparedStatement insertStmt =
conn.prepareStatement(insertSql)) {
                        insertStmt.setString(1, username);
                        insertStmt.setString(2, "withdraw");
                        insertStmt.setDouble(3, amount);
                        insertStmt.executeUpdate();
                    }
                    return "525 OK";  // 操作成功
                } else {
                    return "401 ERROR!";  // 操作失败
                }
            }
        }
    }

    /**
     * 记录服务操作日志
     * @param username 用户名
     * @param serviceName 服务名称
     * @param response 响应代码
     */
    private static void logServiceRecord(String username, String
serviceName, String response) {
        try (Connection conn = DriverManager.getConnection(DB_URL);
             PreparedStatement stmt = conn.prepareStatement(
                     "INSERT INTO service_records (username,
service_name, response_code, timestamp) VALUES (?, ?, ?,
datetime('now'))")) {
            stmt.setString(1, username);
```

```
            stmt.setString(2, serviceName);
            stmt.setString(3, response);
            stmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

4.3 客户端

Client.java 中：

```
package window1;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;

public class Client {
    public static void main(String[] args) {
        try {
            // 创建 Socket 对象，指定服务端的 IP 地址和端口号
            Socket socket = new Socket("192.168.43.232", 2525);
            System.out.println("程序开始运行");

            // 获取输入流和输出流 输入流和输出流是通过 socket 对象来进行数据传输
的。
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(),
true);

            // 从控制台读取用户输入
            BufferedReader reader = new BufferedReader(new
InputStreamReader(System.in));
            String message;

            while (true) {
                message = reader.readLine();
```

```
            if (message.equalsIgnoreCase("exit")) {
                out.println("exit");
                break;
            }

            out.println(message);

            String response = in.readLine();
            System.out.println("服务端响应: " + response);
        }

        // 关闭连接
        socket.close();
    } catch (IOException e) {
        System.err.println("连接失败: " + e.getMessage());
        e.printStackTrace();
    }
  }
}
```

NetworkManager.java 中：

```
package window1;

import java.io.*;
import java.net.*;

public class NetworkManager {
    private static NetworkManager instance;
    private Socket socket;
    private PrintWriter out;
    private BufferedReader in;

    private NetworkManager() {
        try {
            socket = new Socket("192.168.43.232", 2525);
            out = new PrintWriter(socket.getOutputStream(), true);
            in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        } catch (IOException e) {
            System.err.println("连接失败: " + e.getMessage());
            e.printStackTrace();
        }
    }
```

```
    public static synchronized NetworkManager getInstance() {
        if (instance == null) {
            instance = new NetworkManager();
        }
        return instance;
    }

    public void sendMessage(String message) {
        if (out != null) {
            out.println(message);
        }
    }

    public String receiveMessage() {
        if (in != null) {
            try {
                return in.readLine();
            } catch (IOException e) {
                System.err.println("接收失败: " + e.getMessage());
                e.printStackTrace();
            }
        }
        return null;
    }

    public void closeConnection() {
        if (socket != null) {
            try {
                socket.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

ServerTable.java 中：

```
package window1;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class ServeTable extends JFrame {
```

```
    public ServeTable(){
        setTitle("ATM");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        addWindowListener(new WindowAdapter(){
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });

        //创建 gui
        JPanel westPanel = new JPanel();
        JPanel eastPanel = new JPanel();

        //westPanel 作为功能切换板块
        JButton balance = new JButton("Balance");
        JButton withdraw = new JButton("Withdraw");
        JButton exit = new JButton("Exit");
        JButton confirm = new JButton("Confirm");
        //eastPanel 作为显示面板
        JLabel welcomeLabel = new JLabel("Login successfully! ",
JLabel.LEFT);
        JLabel guidingLabel = new JLabel("What do you want to
do?",JLabel.LEFT);

        //设置 label 位置
        eastPanel.setLayout(new GridLayout(2,1,5,5));
        eastPanel.add(welcomeLabel);
        eastPanel.add(guidingLabel);
        //设置按钮位置
        westPanel.setLayout(new GridLayout(3,1,5,5));
        westPanel.add(balance);
        westPanel.add(withdraw);
        westPanel.add(exit);

        add(westPanel,BorderLayout.WEST);
        add(eastPanel,BorderLayout.EAST);
        //设置窗口大小并居中
        setSize(400, 240);
        setResizable(false);
        setLocationRelativeTo(null);
        setVisible(true);

        //功能代码
```

```
        //读取数据


        //添加事件监听器
        balance.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                NetworkManager networkManager =
NetworkManager.getInstance();
                networkManager.sendMessage("BALA");
                String response = networkManager.receiveMessage();

                eastPanel.removeAll();
                JLabel balanceLabel = new JLabel(response,
JLabel.LEFT);
                eastPanel.add(balanceLabel);

                // 重新布局
                eastPanel.revalidate();
                eastPanel.repaint();
                add(eastPanel,BorderLayout.CENTER);
            }
        });
        withdraw.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                //创建布局
                eastPanel.removeAll();
                JTextField withdrawText;
                JLabel withdrawLabel = new JLabel("Enter your withdraw
amount here:");
                withdrawText = new JTextField(20);

                eastPanel.setLayout(new GridLayout(3,1,5,5));
                eastPanel.add(withdrawLabel);
                eastPanel.add(withdrawText);
                eastPanel.add(confirm);

                // 重新布局
                eastPanel.revalidate();
                eastPanel.repaint();
                add(eastPanel,BorderLayout.CENTER);

                confirm.addActionListener(new ActionListener() {
                    public void actionPerformed(ActionEvent ae) {
                        NetworkManager networkManager =
```

```
NetworkManager.getInstance();
                    String withdraw = withdrawText.getText();
                    networkManager.getInstance().sendMessage("WDRA "
+ withdraw);
                    System.out.println("WDRA " + withdraw);
                    String response =
networkManager.receiveMessage();
                    if (response.startsWith("525")) {
                        JOptionPane.showMessageDialog(null, "取款成功!
");
                    }
                    else if(response.startsWith("401")){
                        JOptionPane.showMessageDialog(null, "余额不足!
", "错误", JOptionPane.ERROR_MESSAGE);
                    }
                    else{
                        JOptionPane.showMessageDialog(null, "取款失败!
请联系服务人员！", "错误", JOptionPane.ERROR_MESSAGE);
                    }
                }
            });


        }
    });
    exit.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            NetworkManager networkManager =
NetworkManager.getInstance();
            networkManager.sendMessage("BYE");
            dispose();
        }
    });

}

public static void main(String[] args) {
    new ServeTable();
}
}
```

LoginFrame.java 中：

```
package window1;
```

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class LoginFrame extends JFrame {
    private JPasswordField passwordText;
    private JButton loginButton;

    public LoginFrame() {
        setTitle("ATM");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                NetworkManager.getInstance().closeConnection();
                System.exit(0);
            }
        });

        JPanel panel = new JPanel();
        JPanel topPanel = new JPanel();
        JPanel bottomPanel = new JPanel();

        JLabel passwordLabel = new JLabel("Enter your password
here:");
        passwordText = new JPasswordField(20);

        loginButton = new JButton("Login");
        loginButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent ae) {
                NetworkManager networkManager =
NetworkManager.getInstance();
                String password = new
String(passwordText.getPassword());
//              if (password.equals("")){
//                  JOptionPane.showMessageDialog(null, "密码不能为空，请
重试！", "错误", JOptionPane.ERROR_MESSAGE);
//                  return;
//              }
                networkManager.getInstance().sendMessage("PASS " +
password);
                String response = networkManager.receiveMessage();
                if (response.startsWith("525")) {
                    JOptionPane.showMessageDialog(null, "登录成功！");
                    new ServeTable();
```

```
                dispose();
            } else if(response.startsWith("401")){
                JOptionPane.showMessageDialog(null, "密码错误！", "错
误", JOptionPane.ERROR_MESSAGE);
                passwordText.setText(""); // 清空密码文本框，允许用户重新
输入
            }
        }
    });

    panel.setLayout(new GridLayout(3, 2, 5, 5));
    panel.add(passwordLabel);
    panel.add(passwordText);

    topPanel.setLayout(new FlowLayout(FlowLayout.CENTER));
    JLabel welcomeLabel = new JLabel("Welcome to our bank!",
JLabel.CENTER);
    topPanel.add(welcomeLabel);

    bottomPanel.setLayout(new FlowLayout(FlowLayout.CENTER));
    bottomPanel.add(loginButton);

    add(topPanel, BorderLayout.NORTH);
    add(panel, BorderLayout.CENTER);
    add(bottomPanel, BorderLayout.SOUTH);

    setSize(400, 240);
    setResizable(false);
    setLocationRelativeTo(null);
    setVisible(true);
}

public static void main(String[] args) {
    new LoginFrame();
}
}
```

CardInsertionFrame.java 中：

```
package window1;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
public class CardInsertionFrame extends JFrame {
    private JTextField  usernameText;

    public CardInsertionFrame() {
        setTitle("ATM");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                NetworkManager.getInstance().closeConnection();
                System.exit(0);
            }
        });

        JPanel topPanel = new JPanel();
        JPanel centerPanel = new JPanel();
        JPanel bottomPanel = new JPanel();

        JLabel welcomeLabel = new JLabel("Welcome to our bank!",
JLabel.CENTER);
        topPanel.add(welcomeLabel);

        JLabel usernameLabel = new JLabel("Enter your username
here:");
        usernameText = new JTextField (20);

        centerPanel.setLayout(new GridLayout(3, 2, 5, 5));
        centerPanel.add(usernameLabel);
        centerPanel.add(usernameText);

        JButton okButton = new JButton("OK");
        okButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent ae) {
                NetworkManager networkManager =
NetworkManager.getInstance();
                String username = usernameText.getText();
                if (username.equals("")){
                    JOptionPane.showMessageDialog(null, "账号不能为空，请重
试！", "错误", JOptionPane.ERROR_MESSAGE);
                    return;
                }
                networkManager.getInstance().sendMessage("HELO " +
username);
                System.out.println("HELO " + username);
                String response = networkManager.receiveMessage();
```

```
                if (response.startsWith("500")) {
                    new LoginFrame();
                    dispose();
                } else {
                    JOptionPane.showMessageDialog(null, "账号错误，请重试!
", "错误", JOptionPane.ERROR_MESSAGE);
                    usernameText.setText(""); // 清空用户名文本框，允许用户重
新输入
                }
            }
        });
        bottomPanel.setLayout(new FlowLayout(FlowLayout.CENTER));
        bottomPanel.add(okButton);

        add(topPanel, BorderLayout.NORTH);
        add(centerPanel, BorderLayout.CENTER);
        add(bottomPanel, BorderLayout.SOUTH);

        setSize(400, 240);
        setResizable(false);
        setLocationRelativeTo(null);
        setVisible(true);
    }

    public static void main(String[] args) {
        try {
            // 设置 Nimbus 外观

UIManager.setLookAndFeel("javax.swing.plaf.nimbus.NimbusLookAndFeel")
;
        } catch (Exception e) {
            e.printStackTrace();
        }
        new CardInsertionFrame();
    }
}
```

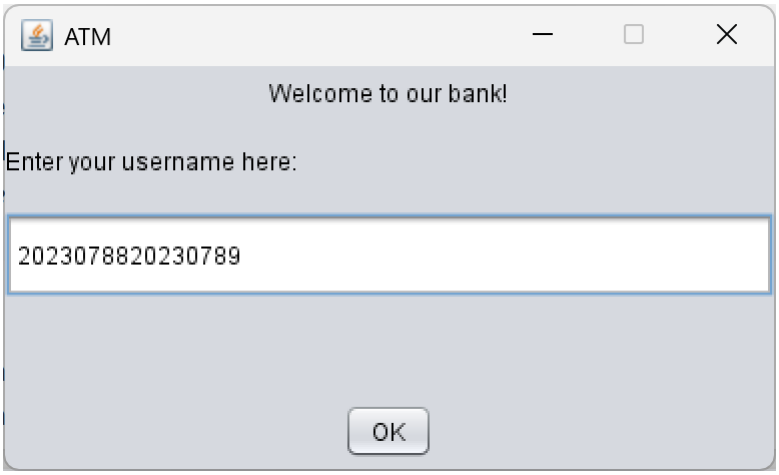
5. 测试运行结果

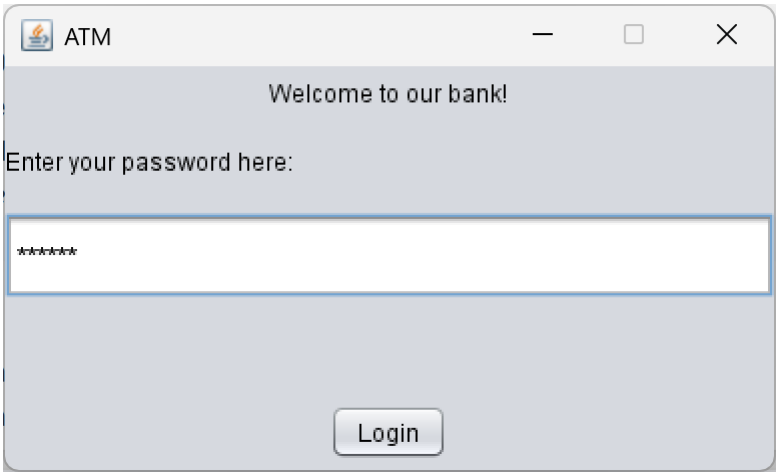
//测试运行过程中的记录，包括截图和文字描述、问题分析、改进方案等。

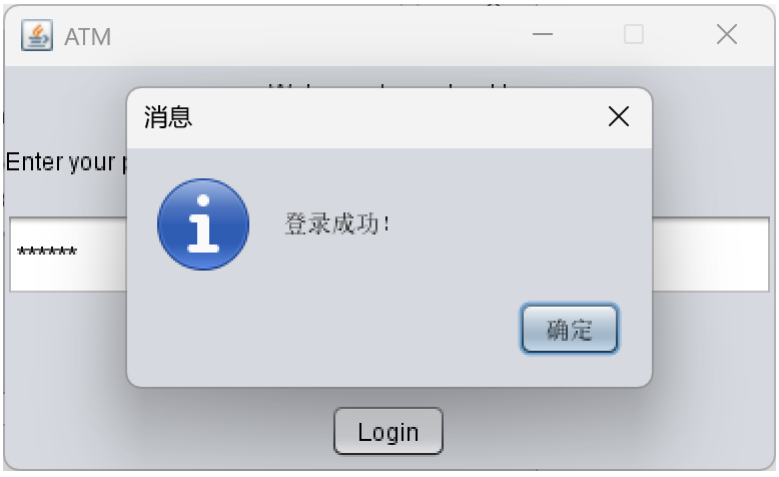
5.1 组内测试

1.客户端截图

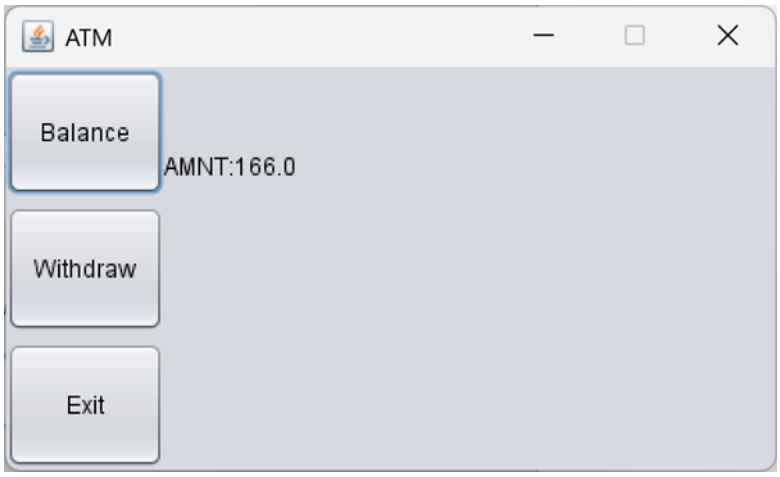
1）成功登录



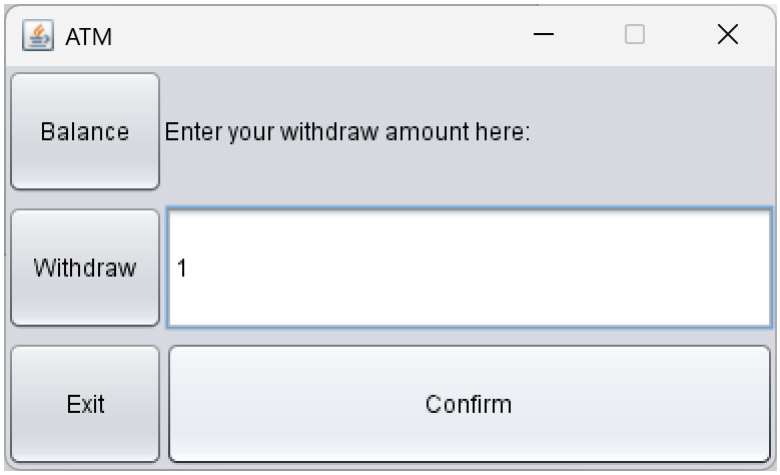




2）查询余额


ATM
Balance
AMNT:166.0
Withdraw
Exit

3）成功取款


ATM
Balance
Enter your withdraw amount here:
Withdraw
1
Exit
Confirm


ATM
Balance
Withdraw
Exit
Confirm
消息
取款成功!
确定

2.服务器端截图：

1）客户端与服务器端之间的报文

```
F:\langu\Java\jdk-11.0.17\bin\java.exe "-javaage
Server started. Listening on port 2525
New client connected: /192.168.90.133
Received command: HELO 2023078820230789
Received command: PASS 123456
Received command: BALA
Received command: WDRA 1
Received command: BALA
Received command: BYE
```

2）数据库的操作记录

143	2023078820230789	PASS	NULL	525 OK!	2025-05-11 12:05:56
144	2023078820230789	BALA	NULL	AMNT:166.0	2025-05-11 12:06:02
145	2023078820230789	withdraw	1	525 OK	2025-05-11 12:06:21
146	2023078820230789	WDRA	NULL	525 OK	2025-05-11 12:06:21
147	2023078820230789	BALA	NULL	AMNT:165.0	2025-05-11 12:06:29
148	2023078820230789	BYE	NULL	BYE	2025-05-11 12:06:40

3）数据库的最后测试结果

	username	passworc	balance
1	2023078820230789	123456	165
2	user2	pass2	500
3	user3	pass3	2000

5.2 组间测试

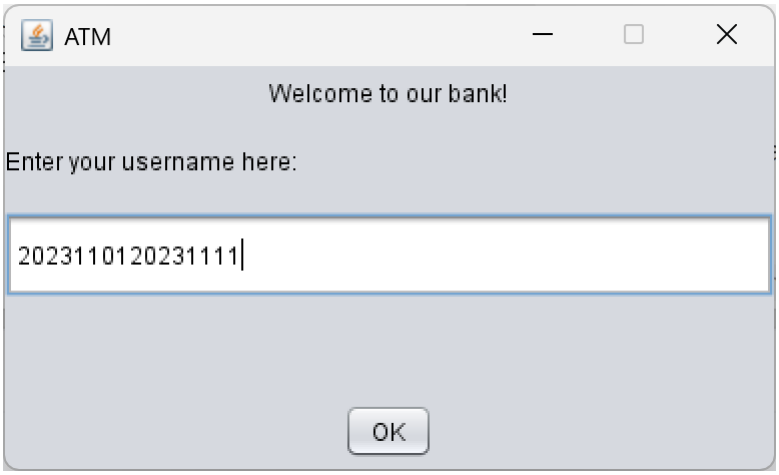
一、和农炯斌/刘航小组的测试

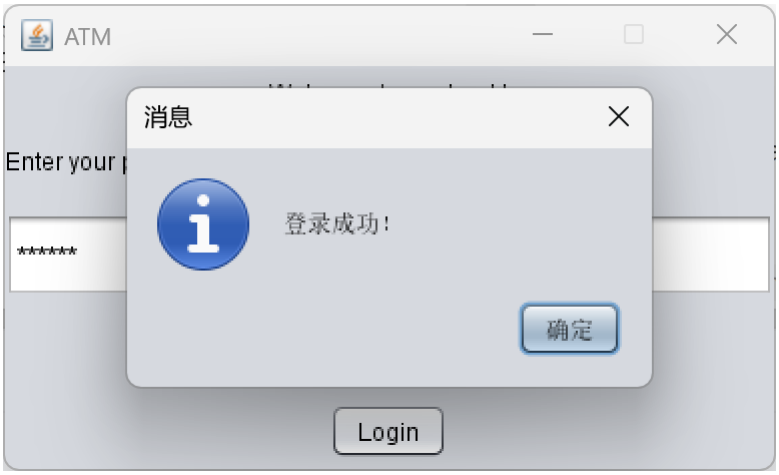
9	农炯斌20231111	刘航20231101	2023110120231111	123456	¥ 100,000.00

我方作为 Client，对方作为 Server

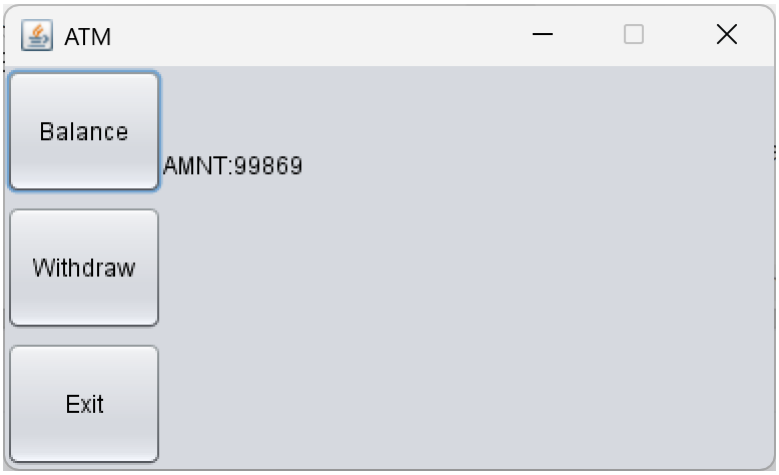
1.客户端截图

1）成功登录

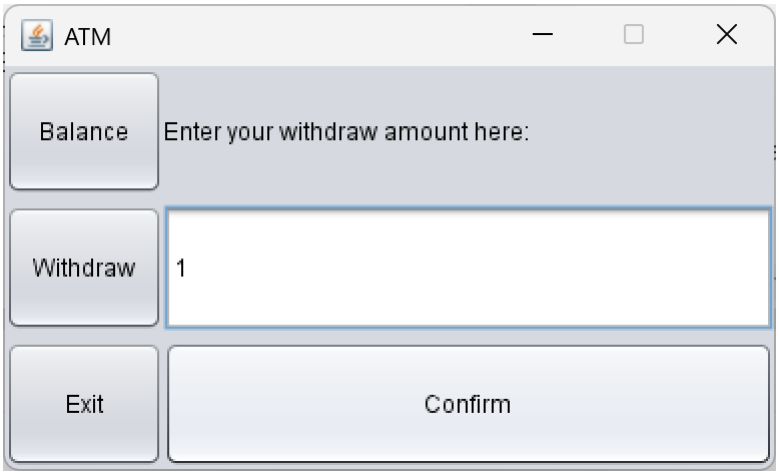


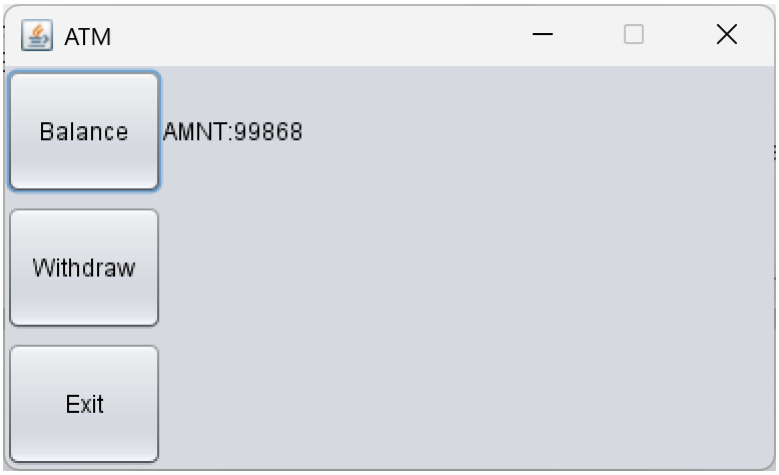


2)查询余额



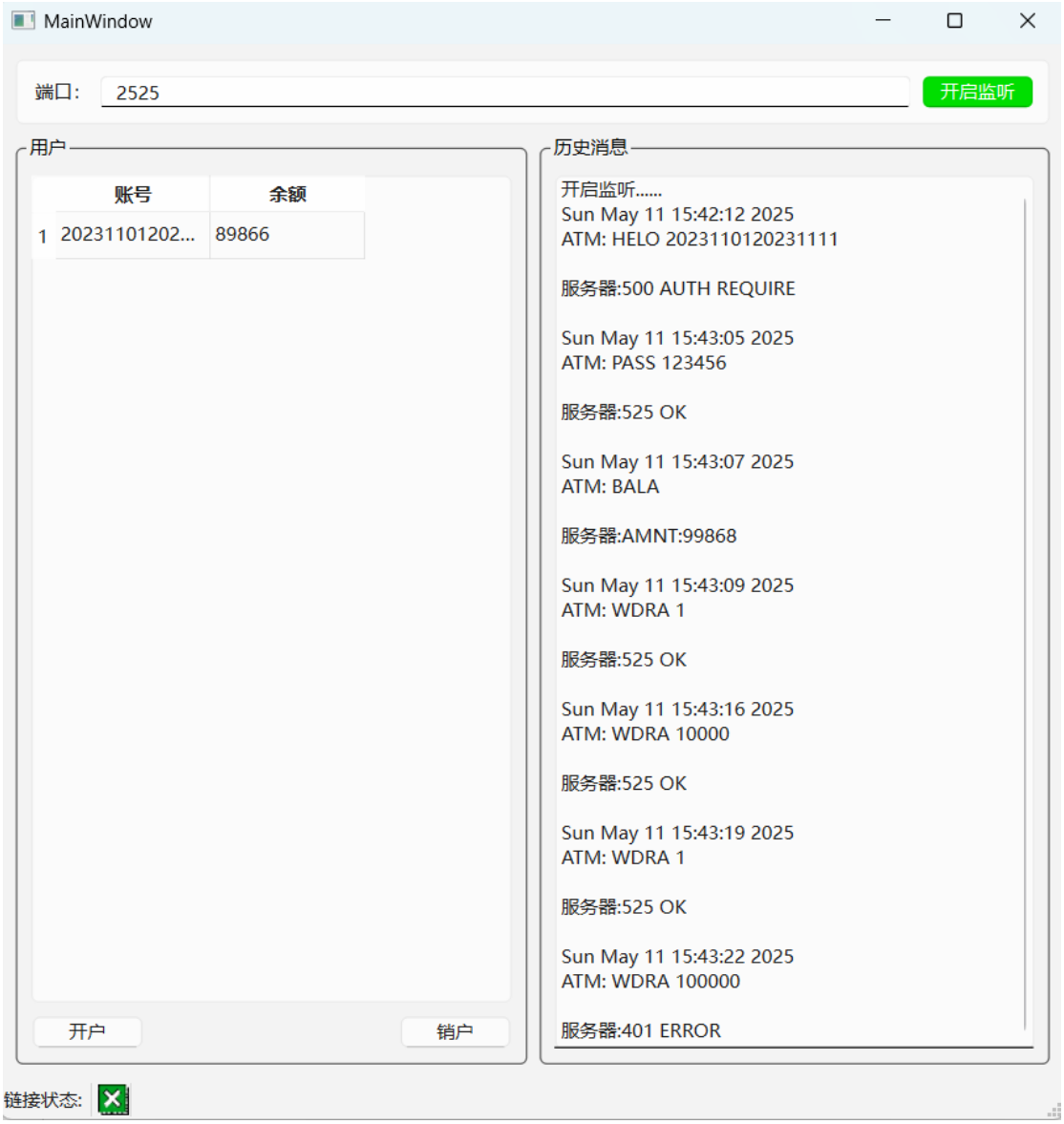
3）成功取款



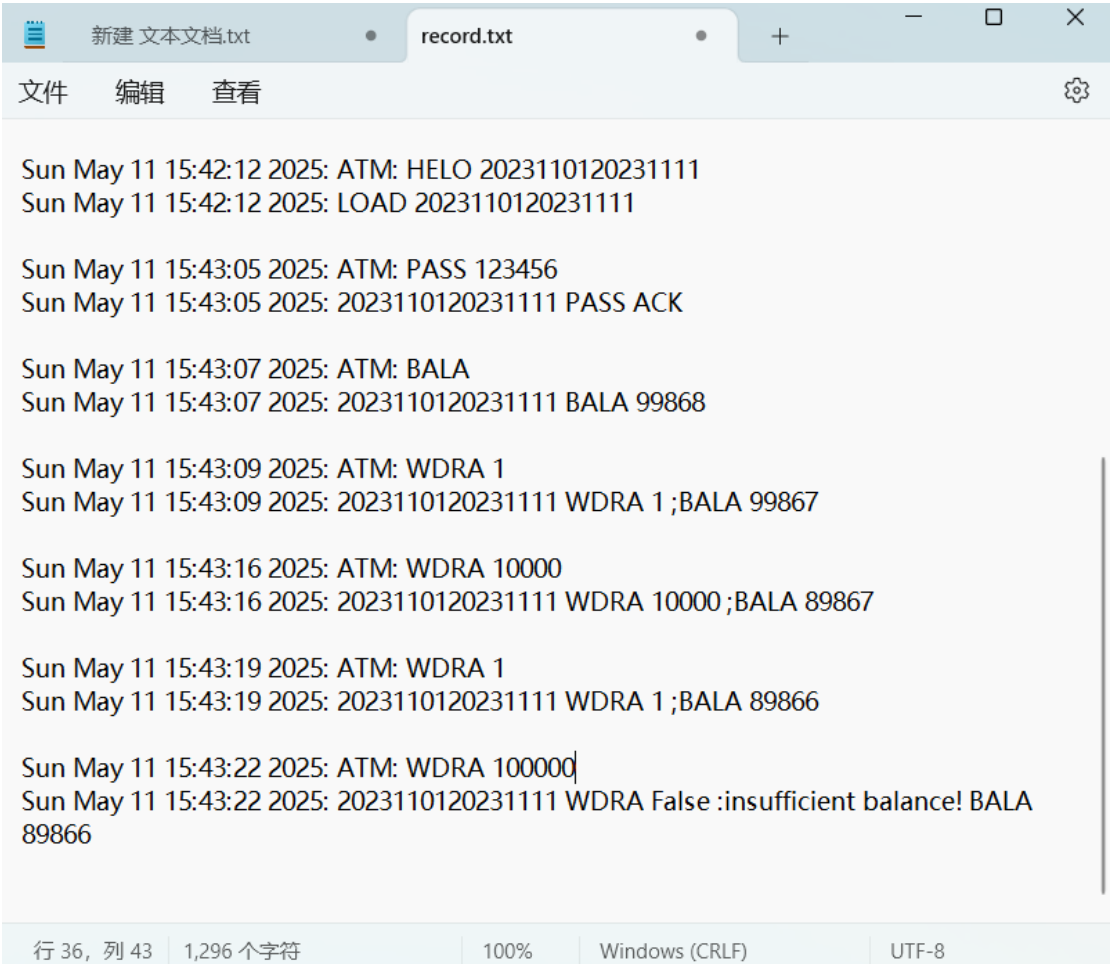


2.服务器端截图

1）客户端和服务器端之间的报文


MainWindow
端口:
2525
开启监听
用户
账号
余额
1
20231101202...
89866
开户
销户
历史消息
开启监听.....
Sun May 11 15:42:12 2025
ATM: HELO 2023110120231111
服务器:500 AUTH REQUIRE
Sun May 11 15:43:05 2025
ATM: PASS 123456
服务器:525 OK
Sun May 11 15:43:07 2025
ATM: BALA
服务器:AMNT:99868
Sun May 11 15:43:09 2025
ATM: WDRA 1
服务器:525 OK
Sun May 11 15:43:16 2025
ATM: WDRA 10000
服务器:525 OK
Sun May 11 15:43:19 2025
ATM: WDRA 1
服务器:525 OK
Sun May 11 15:43:22 2025
ATM: WDRA 100000
服务器:401 ERROR
链接状态:

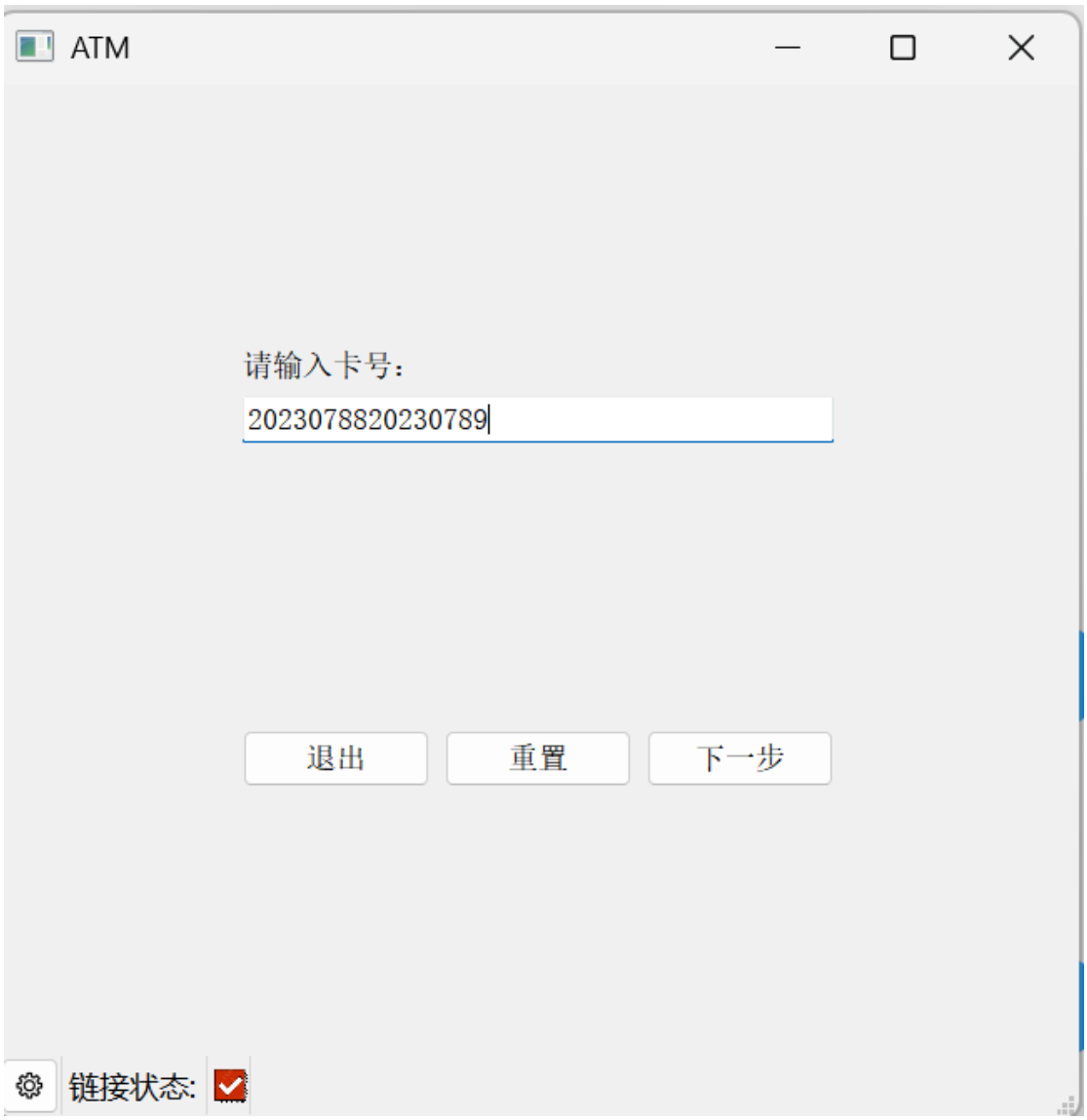
2）数据库的操作记录

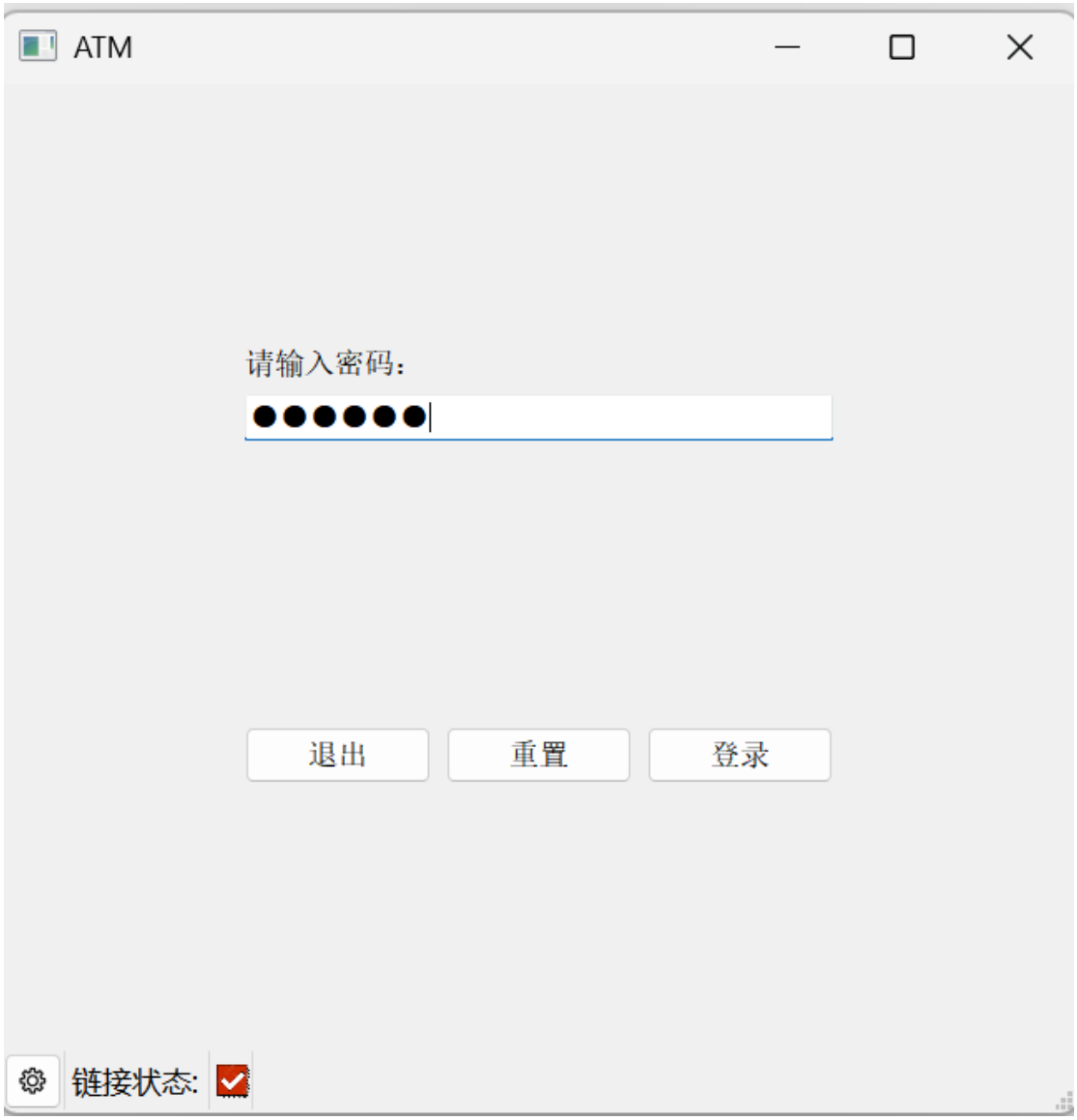


我方作为 Server，对方作为 Client

1.客户端截图

1）成功登录


ATM
请输入卡号：
2023078820230789
退出
重置
下一步
链接状态:



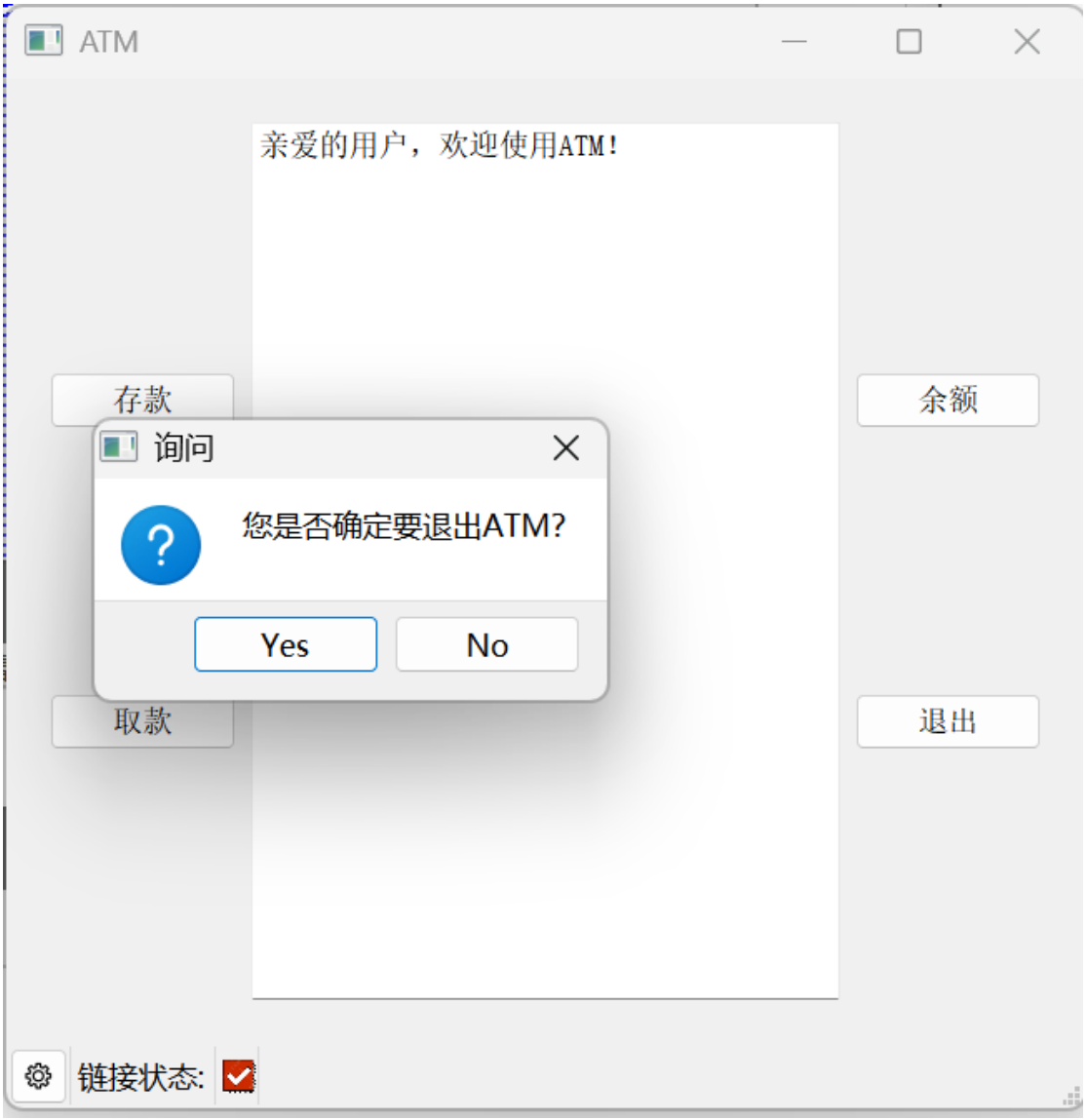
2）查询余额

3）成功取款


ATM
100
1000
请输入取款金额：
500
Info
余额不足!
200
2000
OK
500
3000
退出
重置
确定
800
5000
链接状态:


ATM
100
1000
请输入取款金额:
200
Info
取款成功!
OK
200
2000
500
3000
退出
重置
确定
800
5000
链接状态:

4）成功退出



2.服务器端截图

1）客户端与服务器端之间的报文

```
New client connected: /10.244.35.191
Received command: HELO 2023078820230789
Received command: PASS 123456
Received command: BALA
Received command: WDRA 200
Received command: BALA
Received command: WDRA 500
Received command: WDRA 500
Received command: BYE
```

2）数据库的操作记录

133	2023078820230789	PASS	NULL	525 OK!	2025-05-11 07:31:12
134	2023078820230789	BALA	NULL	AMNT:366.0	2025-05-11 07:31:13
135	2023078820230789	withdraw	200	525 OK	2025-05-11 07:31:41
136	2023078820230789	WDRA	NULL	525 OK	2025-05-11 07:31:41
137	2023078820230789	BALA	NULL	AMNT:166.0	2025-05-11 07:31:52
138	2023078820230789	WDRA	NULL	401 ERROR!	2025-05-11 07:32:07
139	2023078820230789	WDRA	NULL	401 ERROR!	2025-05-11 07:32:22
140	2023078820230789	BYE	NULL	BYE	2025-05-11 07:32:31

3）数据库的最后测试结果

	username	passworc	balance
1	2023078820230789	123456	166
2	user2	pass2	500
3	user3	pass3	2000

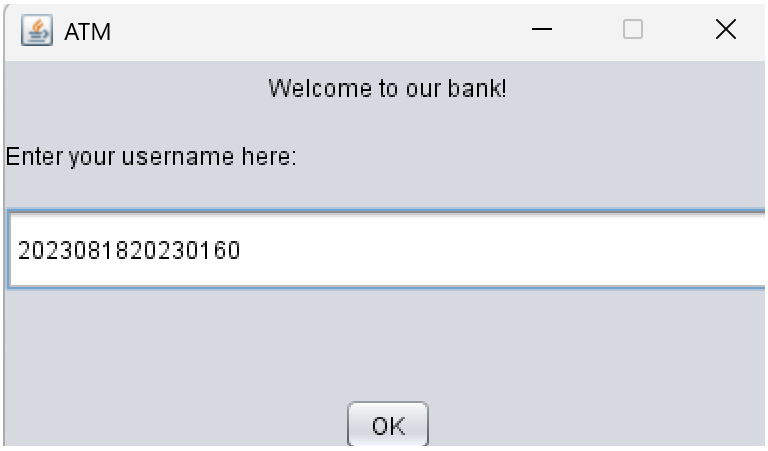
二、和冯子芮/雷佳俐小组的测试

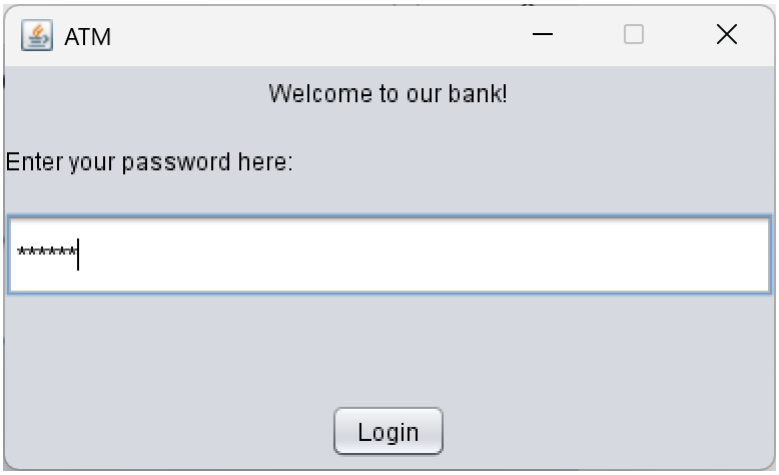
17	冯子芮20230818	雷佳俐20230160	2023081820230160	123456	￥10,000.0

我方作为 Client，对方作为 Server

1.客户端截图

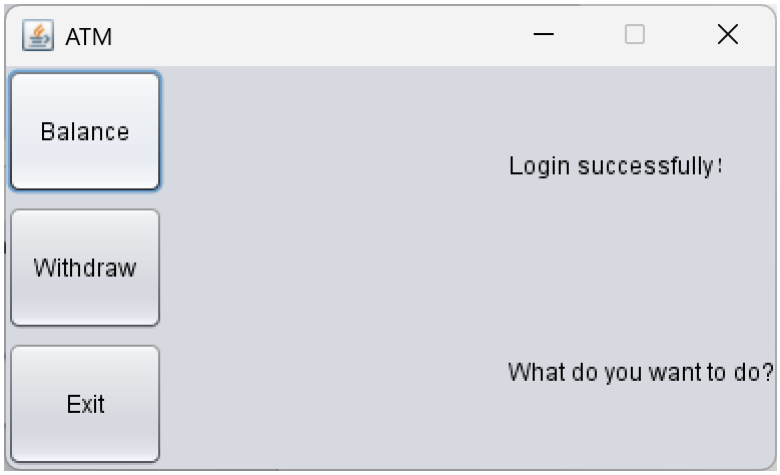
1）成功登录


ATM
Welcome to our bank!
Enter your username here:
2023081820230160
OK


ATM
Welcome to our bank!
Enter your password here:
******
Login

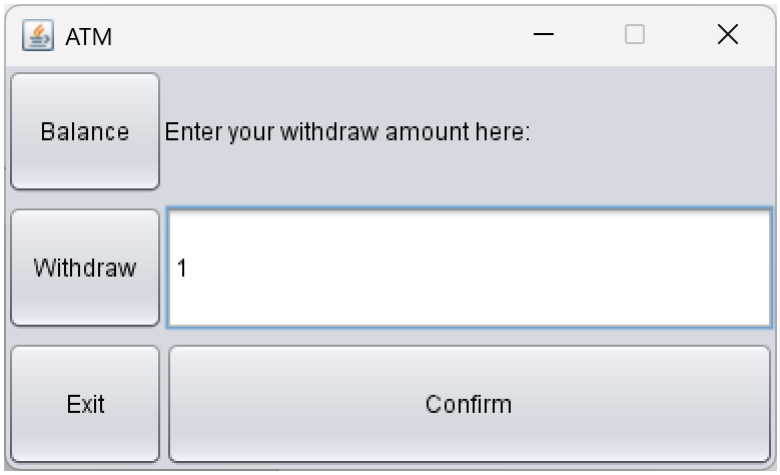

ATM
消息
登录成功!
确定
Login

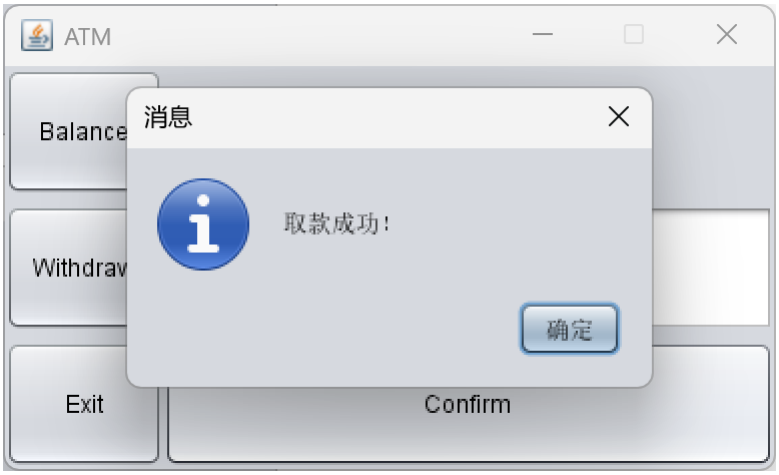
2）查询余额

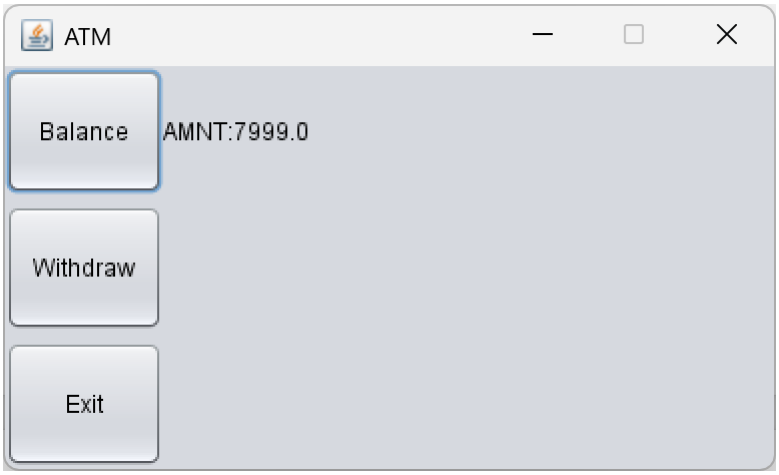

ATM
Balance
Withdraw
Exit
Login successfully!
What do you want to do?


ATM
Balance
Withdraw
Exit
AMNT:8000.0

3）成功取款


ATM
Balance
Withdraw
Exit
Enter your withdraw amount here:
1
Confirm

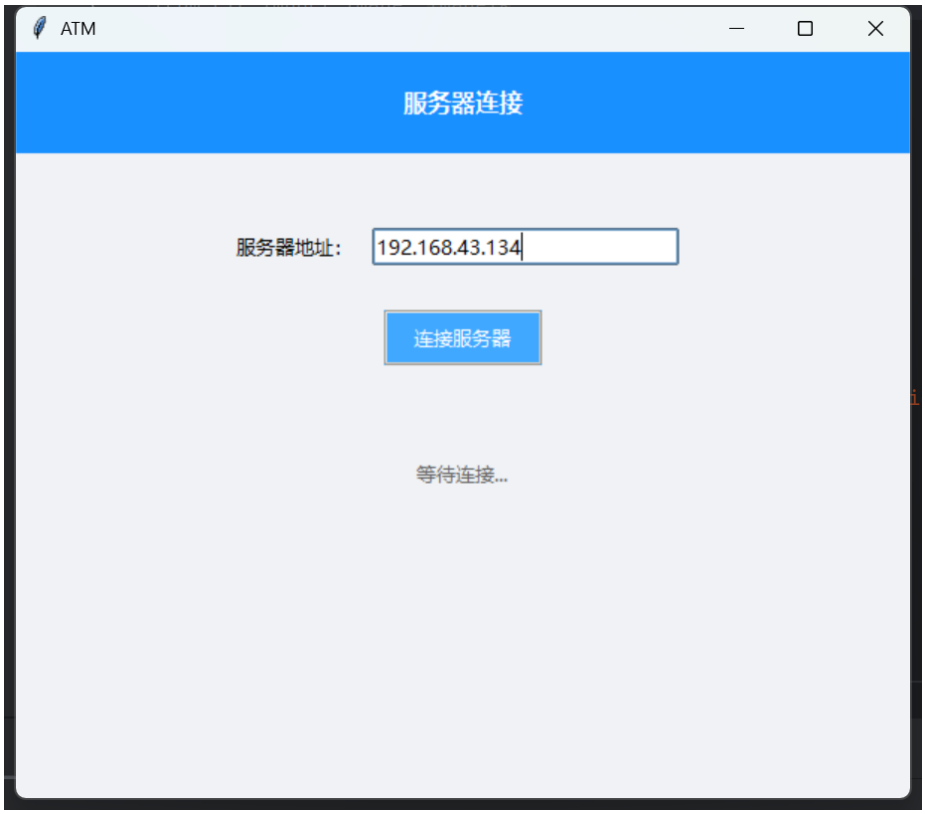


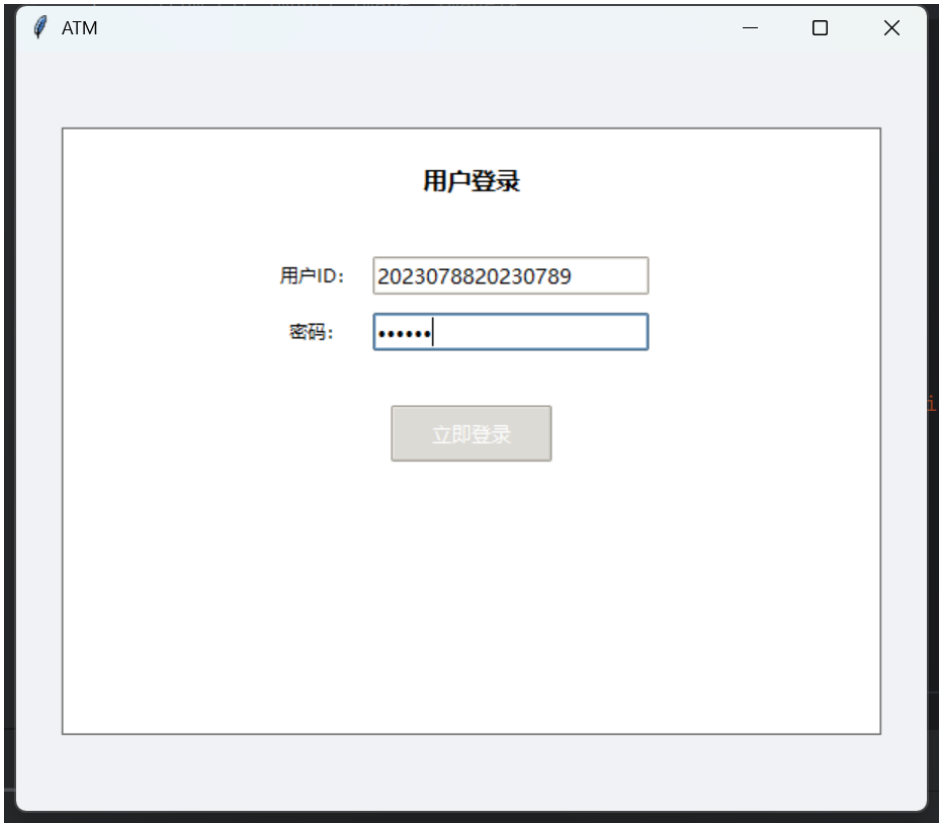


我方作为 Server，对方作为 Client

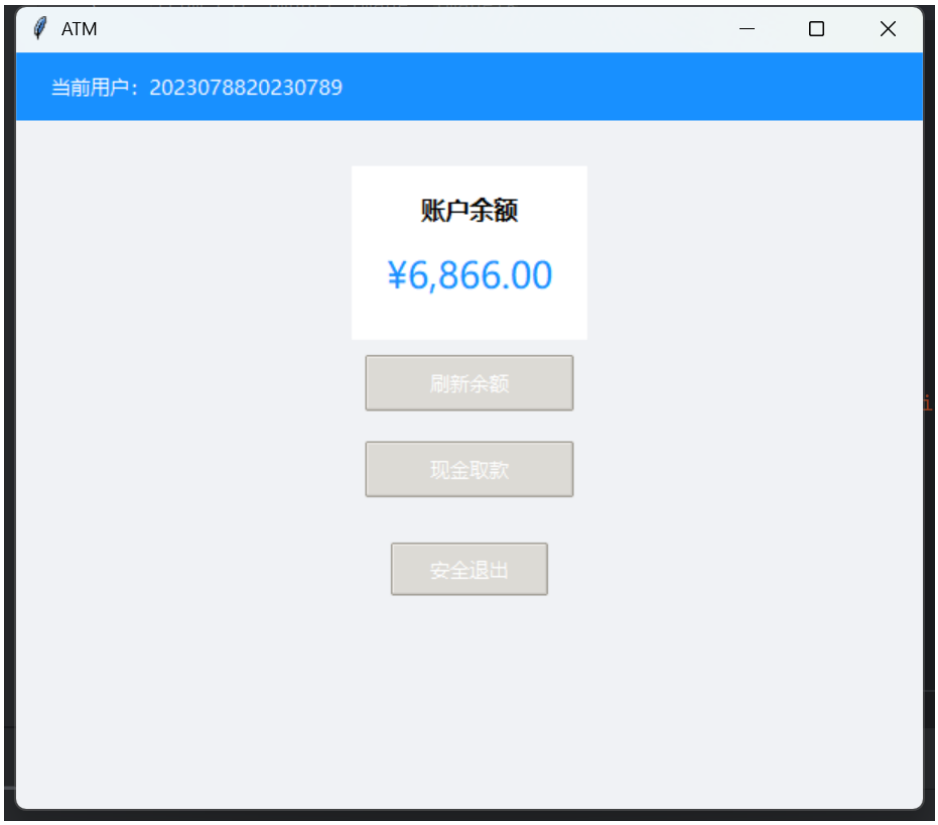
1.客户端截图

1）成功登录

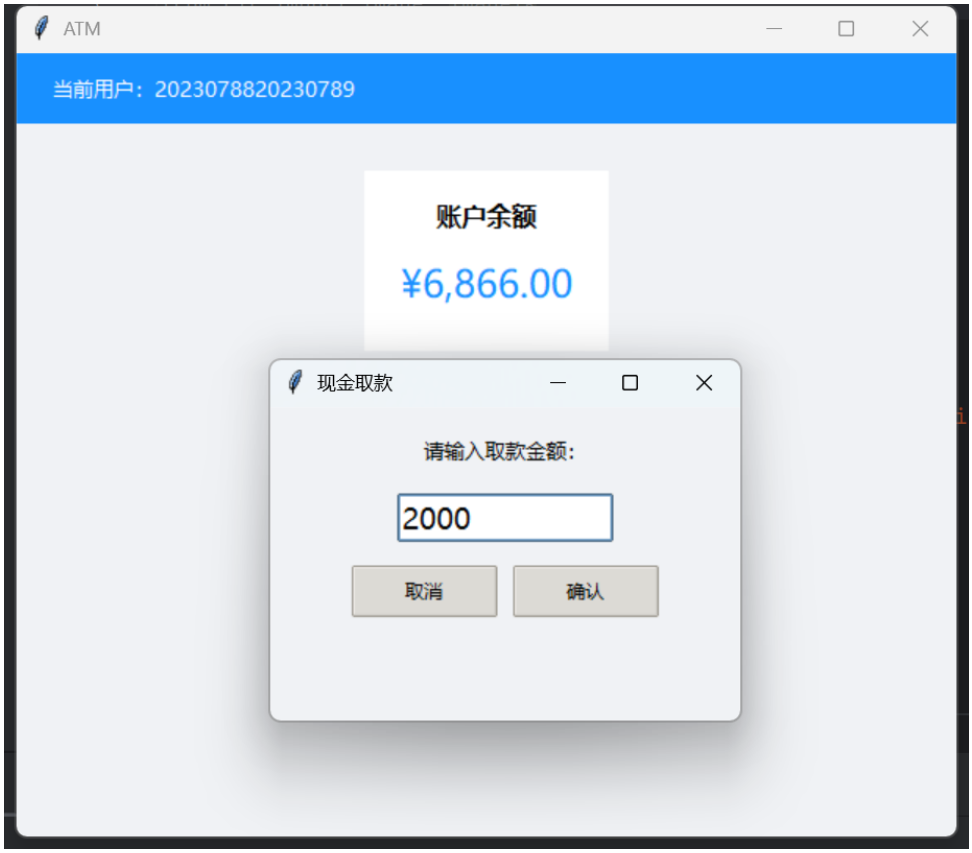

ATM
服务器连接
服务器地址：
192.168.43.134
连接服务器
等待连接...

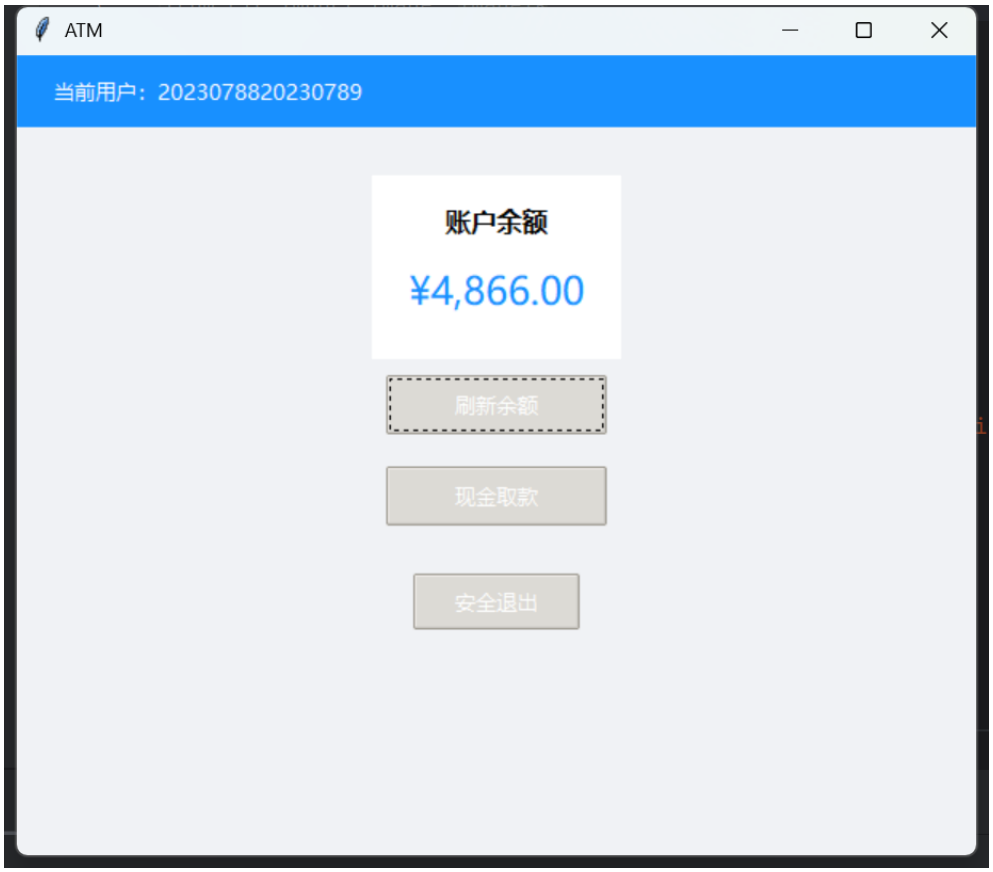

ATM
用户登录
用户ID：
2023078820230789
密码：
立即登录

2）查询余额


ATM
当前用户：2023078820230789
账户余额
¥6,866.00
刷新余额
现金取款
安全退出

3）成功取款


ATM
当前用户：2023078820230789
账户余额
¥6,866.00
现金取款
请输入取款金额：
2000
取消
确认


ATM
当前用户：2023078820230789
账户余额
¥4,866.00
刷新余额
现金取款
安全退出

4）断开连接


ATM
服务器连接
服务器地址：
192.168.43.134
连接服务器
连接已断开

2.服务器端截图

1）客户端和服务器端之间的报文

```
F:\langu\Java\jdk-11.0.17\bin\java.exe "-javaagent:F:\
Server started. Listening on port 2525
New client connected: /192.168.43.246
Received command: HELO 2023078820230789
Received command: PASS 123456
Received command: BALA
Received command: WDRA 2000.0
Received command: BALA
Received command: BYE
```

2）数据库的操作记录

75	2023078820230789	PASS	NULL	525 OK!	2025-05-06 12:31:28
76	2023078820230789	BALA	NULL	AMNT:9997.0	2025-05-06 12:31:28
77	2023078820230789	withdraw	3000	525 OK	2025-05-06 12:31:40
78	2023078820230789	WDRA	NULL	525 OK	2025-05-06 12:31:40
79	2023078820230789	withdraw	30	525 OK	2025-05-06 12:31:47
80	2023078820230789	WDRA	NULL	525 OK	2025-05-06 12:31:47
81	2023078820230789	withdraw	1	525 OK	2025-05-06 12:32:06
82	2023078820230789	WDRA	NULL	525 OK	2025-05-06 12:32:06

3）数据库的最后测试结果

username	passworc	balance
2023078820230789	123456	4866
user2	pass2	500
user3	pass3	2000

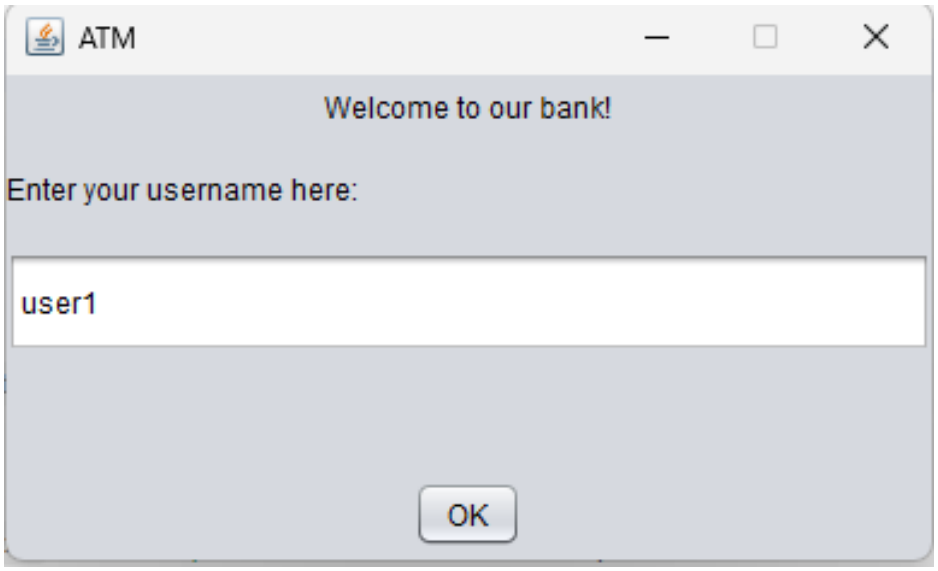
三、和

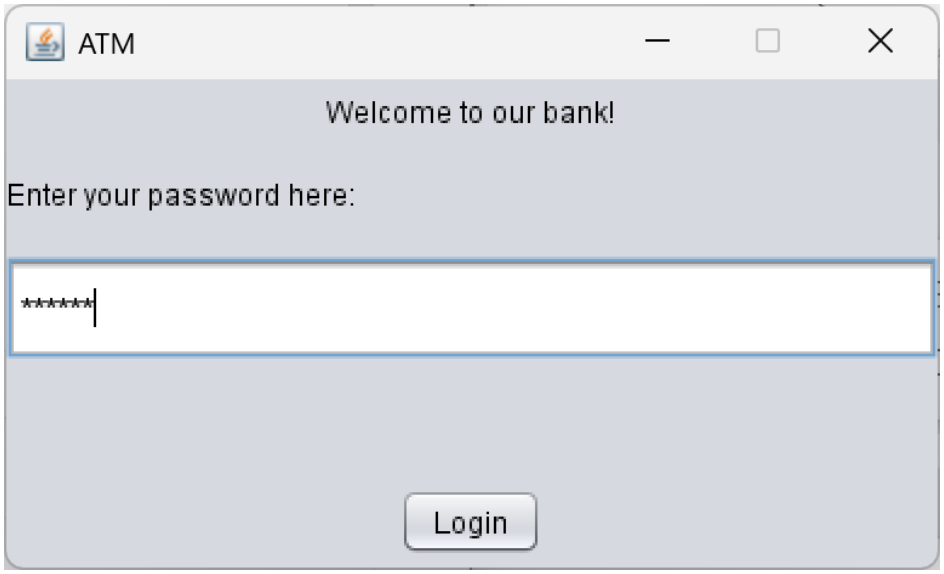
（对方小组的截图）

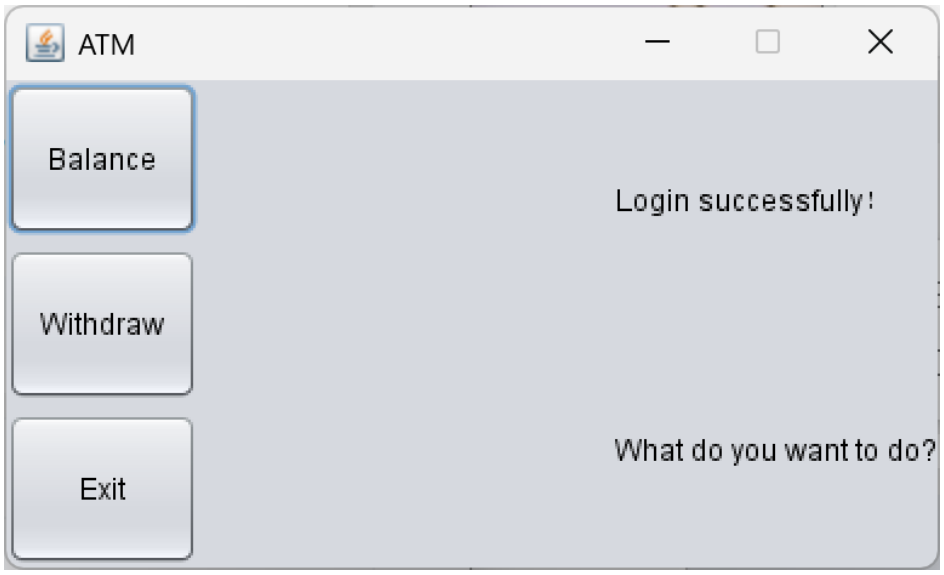
我方作为 Client，对方作为 Server

1.客户端截图

1）成功登录


ATM
Welcome to our bank!
Enter your username here:
user1
OK

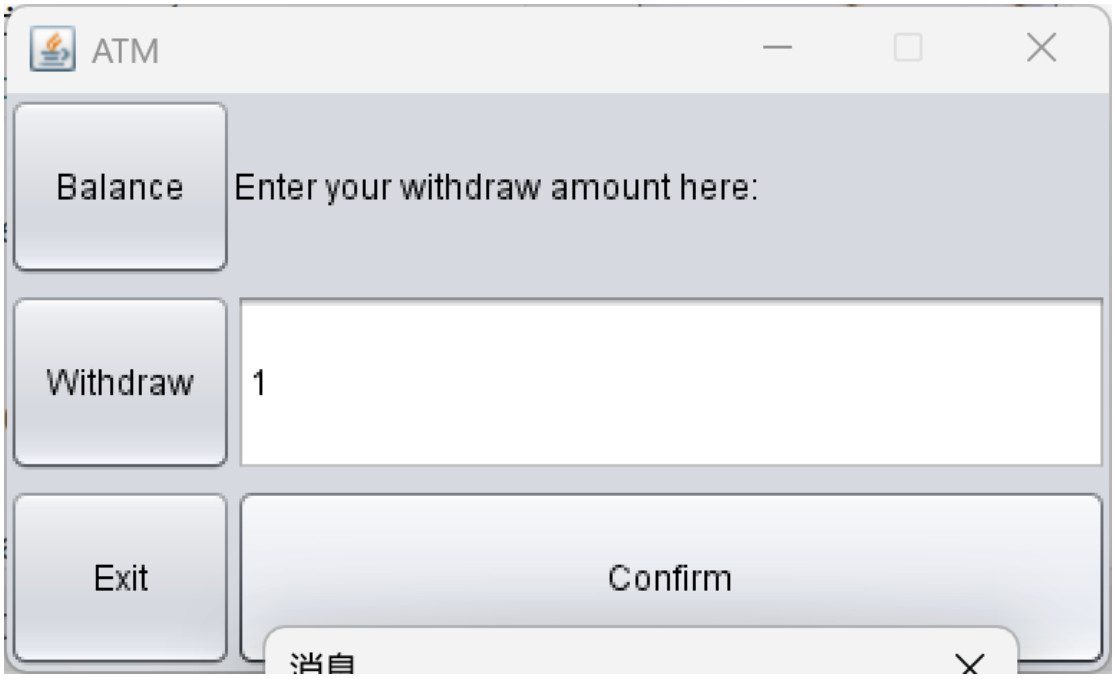

ATM
Welcome to our bank!
Enter your password here:
******
Login

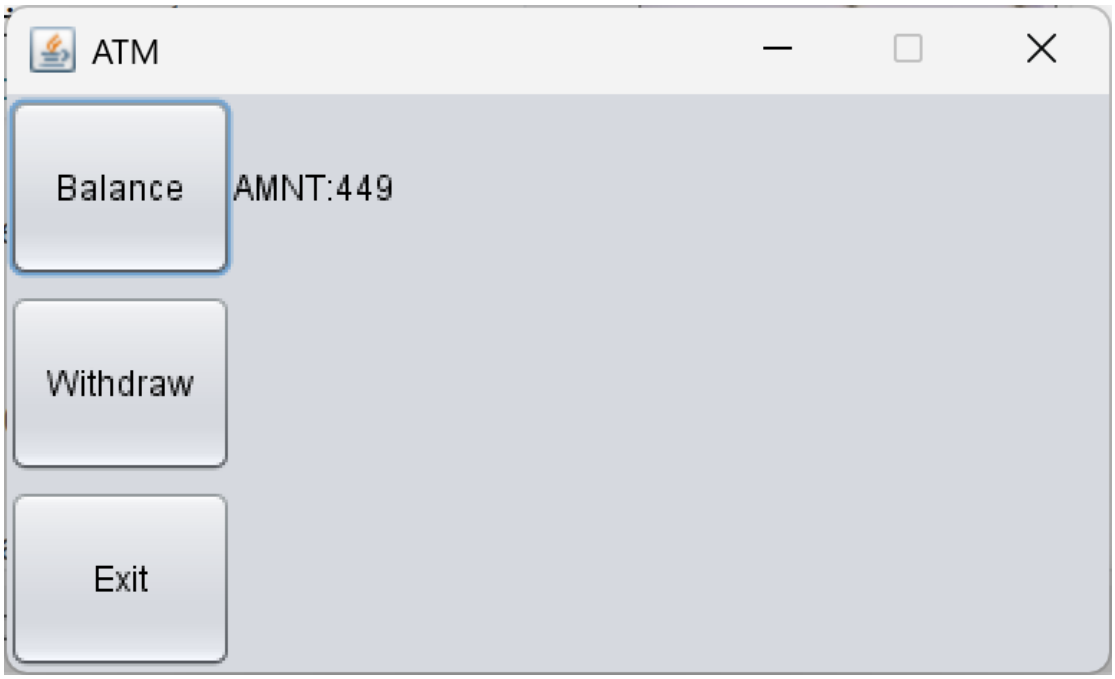

ATM
Balance
Withdraw
Exit
Login successfully!
What do you want to do?

2）查询余额


ATM
Balance
AMNT:450
Withdraw
Exit

3）成功取款


ATM
Balance
Enter your withdraw amount here:
Withdraw
1
Exit
Confirm



2.服务器端截图

1）客户端与服务器端之间的报文

```
2025-05-17 20:46:41,518 INFO:Server is listening...
2025-05-17 20:46:49,474 INFO:New connection from ('10.244.103.119', 62192)
2025-05-17 20:46:49,474 INFO:Received: HELO user1
2025-05-17 20:46:49,474 INFO:Sent: 500 AUTH REQUIRE
2025-05-17 20:46:54,629 INFO:Received: PASS 123456
2025-05-17 20:46:54,630 INFO:Sent: 525 OK!
2025-05-17 20:47:11,785 INFO:Received: BALA
2025-05-17 20:47:11,785 INFO:Sent: AMNT:450
2025-05-17 20:47:18,645 INFO:Received: WDRA 1
2025-05-17 20:47:18,647 INFO:Sent: 525 OK!
2025-05-17 20:47:32,470 INFO:Received: BALA
2025-05-17 20:47:32,470 INFO:Sent: AMNT:449
2025-05-17 20:47:44,553 INFO:Received: BYE
```

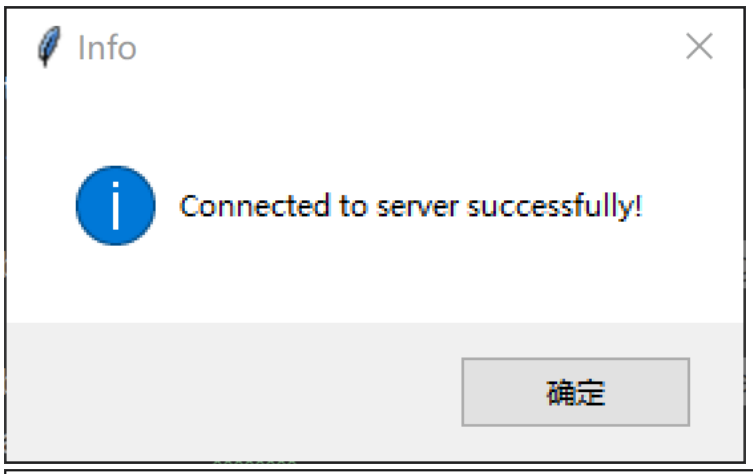
2）数据库的最后测试结果

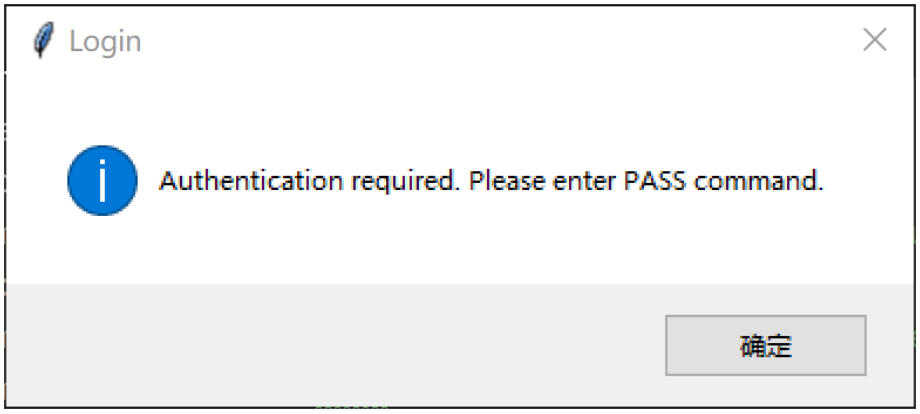
```
{
    "user1": {
        "password": "123456",
        "balance": 449
    },
    "user2": {
        "password": "abcdef",
        "balance": 4900
    },
    "user3": {
        "password": "qwerty",
        "balance": 2000
    }
}
```

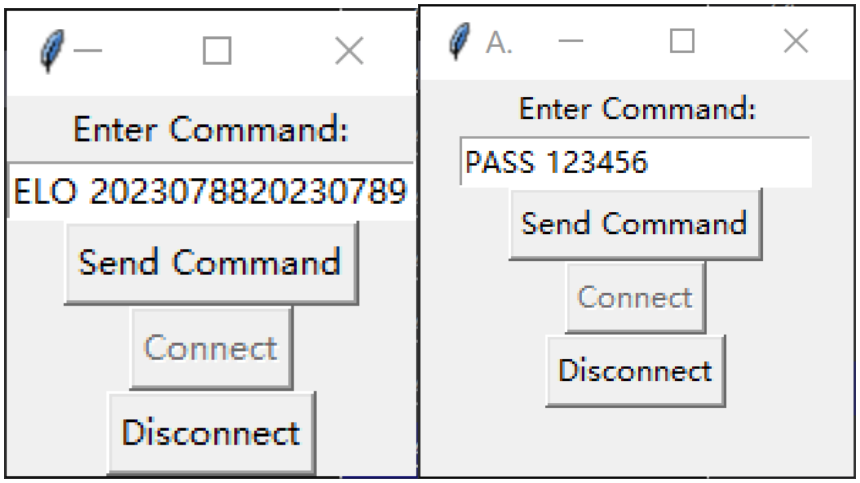
我方作为 Server，对方作为 Client

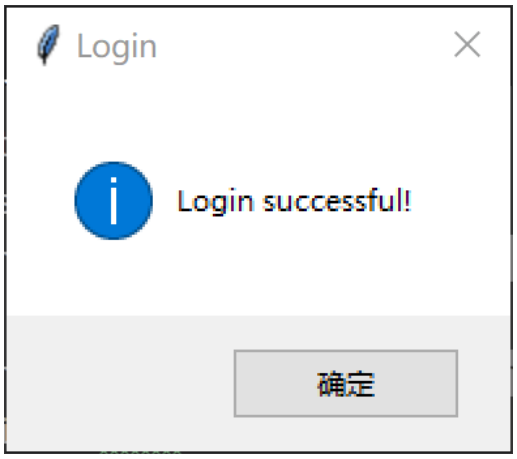
1.客户端截图

1）成功登录

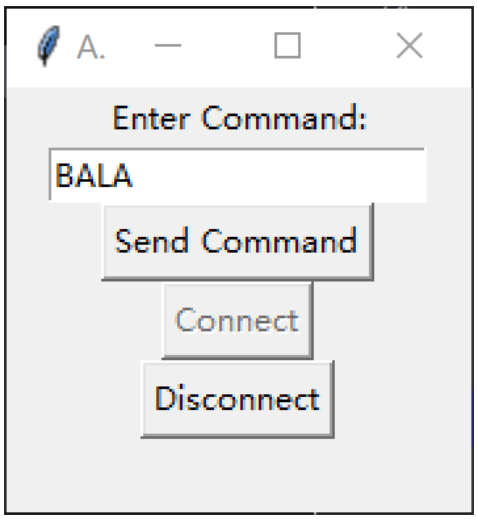

Info
Connected to server successfully!
确定


Login
Authentication required. Please enter PASS command.
确定

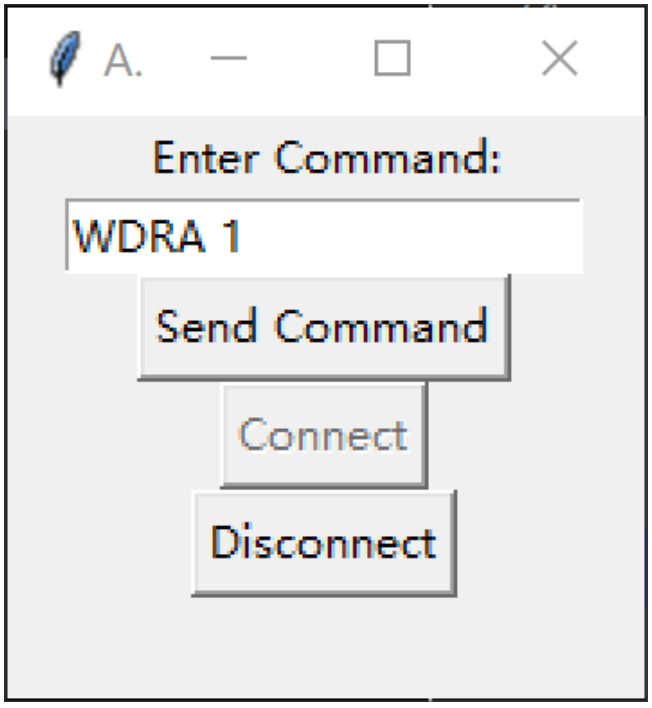

Enter Command:
ELO 2023078820230789
Send Command
Connect
Disconnect
A.
Enter Command:
PASS 123456
Send Command
Connect
Disconnect

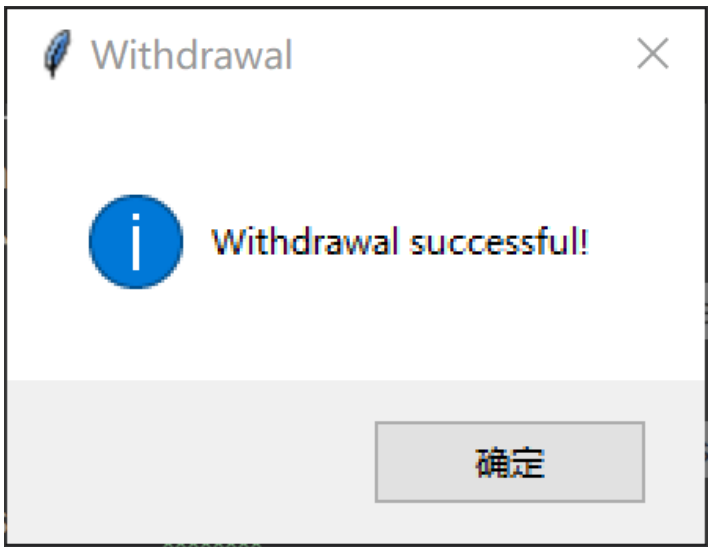

Login
Login successful!
确定

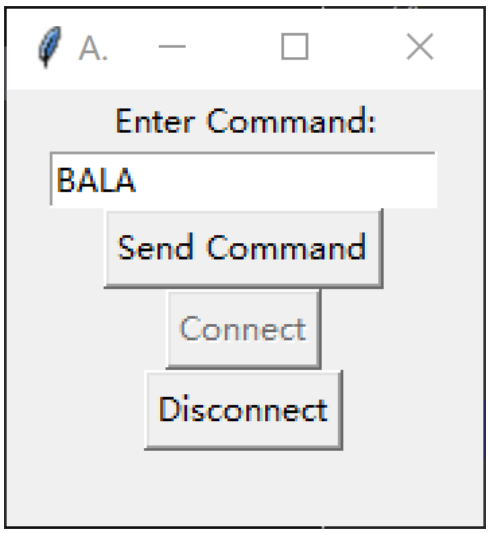
2）查询余额

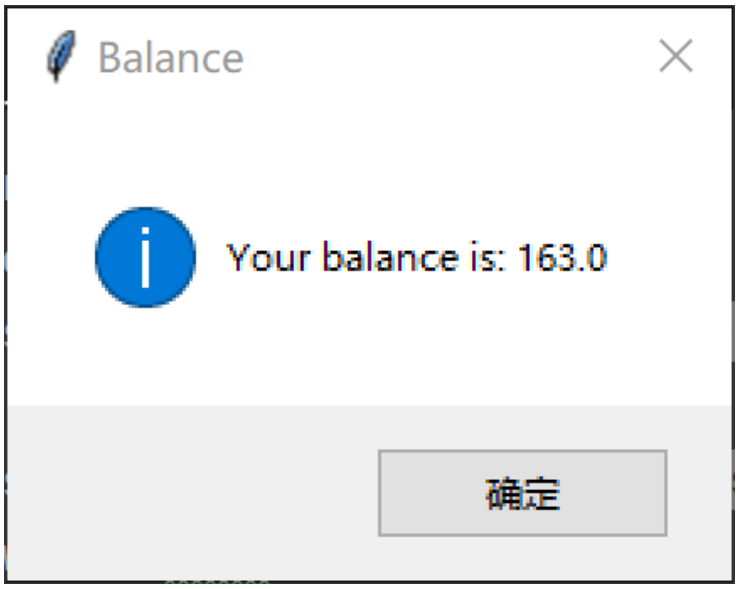

A.
Enter Command:
BALA
Send Command
Connect
Disconnect

3）成功取款


A.
Enter Command:
WDRA 1
Send Command
Connect
Disconnect







2.服务器端截图

1）数据库的操作记录

150	2023078820230789	PASS	*NULL*	525 OK!	2025-05-17 13:09:01
151	2023078820230789	BALA	*NULL*	AMNT:165.0	2025-05-17 13:09:09
152	2023078820230789	PASS	*NULL*	525 OK!	2025-05-17 13:10:34
153	2023078820230789	withdraw	1	525 OK	2025-05-17 13:10:41
154	2023078820230789	WDRA	*NULL*	525 OK	2025-05-17 13:10:41
155	2023078820230789	PASS	*NULL*	525 OK!	2025-05-17 13:13:00
156	2023078820230789	BALA	*NULL*	AMNT:164.0	2025-05-17 13:13:23
157	2023078820230789	withdraw	1	525 OK	2025-05-17 13:13:43
158	2023078820230789	WDRA	*NULL*	525 OK	2025-05-17 13:13:43
159	2023078820230789	BALA	*NULL*	AMNT:163.0	2025-05-17 13:14:27

2）数据库的最后测试结果

	username	passworc	balance
1	2023078820230789	123456	163
2	user2	pass2	500
3	user3	pass3	2000

6. 总结

1.在作业二的时候，组间测试有时候会出现错误，分析发现是大家没有完全按照原来的协议进行命令的编写，存在一些小错误，比如“ ”写成“sp”等错误，或者是协议前后描述不一致（有空格或者是无空格）。

解决思路：

（1）统一协议前后对于命令的规定。

（2）检查大家的源代码是否完全和协议匹配

收获：

在进行网络通信时，协议必须严格制定，各通信方必须严格遵守协议。

2.和使用不同语言编写的客户端/服务器端进行测试的时候，发现因为不同语言之间关于自动换行的定义不一样所以会出现错误。（比如 java 自带换行符而 python 不自带换行符）

解决思路：

（1）让使用自带换行符的语言编写的端系统删除换行符，或者让自身没有带换行符的一端加上换行符

（2）可以在以后的代码编写中加入针对不同语言的连接模块以保证语言兼容性